INTERSYSTEMS · 開発者コミュニティ LT
10 MIN LIGHTNING TALK

FORMAT
LT 10 min

AI AGENT, AUTHORED BY AI

AI エージェントは、AI に作らせる。

IRIS AI Hub × iris-agentic-dev

日本語でお願いするだけで、Agent が組み上がる 6 分

STACK

IRIS 2027.1 AI | iris-agentic-dev v1.6.2 | Claude Code (MCP) | Clinosim JP cohort | OpenAI gpt-4o

TOTAL CODE WRITTEN BY HUMAN

0

lines · ただし日本語プロンプト 1 本

徳山 友雄 · InterSystems Japan · 2026

IRIS 開発者あるある、ありません?

3 つの「痛み」— AI 支援で全部軽くなります

ASK YOURSELF
1 日に何回繰り返してる?

1

クラスリファレンス
1 日に何回開く?

クラス
リファレンス | %Status | $ListBuild | %Dictionary | ...

気がつくと別窓で 5 個開いてる

EXAMPLE

「%Status の作法確認...」
「$ListBuild の挙動...」
「あのメソッドの引数順...」

✓ **AI が全部覚えてて、代わりに引く**

2

$Piece ネスト、
読める?新人に説明できる?

```
;; 典型的な一行:
Set name = $Piece($Piece(
$ListGet(data,1),"^",2),
",",1)
;; 1 時間後、何だっけ...
```

PAIN POINT

書くのも読むのも時間がかかる
3 ヶ月後の自分も理解できない
新人への説明に 1 時間

✓ **AI が最初から避ける書き方を選ぶ**

3

Production 新人教育、
何時間かかります?

Service | Process | Operation | Msg Routing
DTL | Biz Rule | Lookup | BPL | HL7 Schema

これ全部、ゼロから覚えてもらう...

COST

OJT 1-2 ヶ月は当たり前
覚えるべき GUI エディタ 5+
設計意図が暗黙知化しがち

✓ **「Service 追加して」で XData 生成**

TODAY'S PROMISE

その痛み、全部 AI 支援で軽くなります。

しかも **エディタ乗り換え不要** — Studio の横の別ターミナル、VS Code の統合ターミナル、どちらでも OK。

INSTALL
10 min
brew + mcp add + toml

本日のゴール: この 3 つの痛みを、6 分のライブデモで解消できることを体感してもらう

%AI.* クラス一覧

IRIS 2027.1 AI build 103 · 61 クラス · 16 パッケージ

LT DEMO で触る 6 クラス

① ② ③ ④ ⑤ ⑥

Agent 4

① %AI.Agent

① %AI.Agent.Session

%AI.Agent.Skill

%AI.Agent.SubAgent

エージェント本体 + 会話/tool履歴

Tool / ToolMgr 6

② %AI.Tool

%AI.Tool.Generator

%AI.Tool.Resolver

%AI.Tool.Schema

%AI.Tool.StdioHandler

%AI.ToolMgr

docstring/signature → tool spec 自動変換

ToolSet 19

③ %AI.ToolSet

+ 18 Specification サブクラス:

Compiler · Config · EnvVar · Exclude
FastMCP · Filter · Include · Node
MCP · MCP.Remote · MCP.Stdio
Policies · PolicyElement · Query
Requirement · Tool · ToolSet · Utils

XData <Include> で宣言的集約

Policy 5

%AI.Policy.Audit

%AI.Policy.Authorization

⑤ %AI.Policy.ConsoleAudit

%AI.Policy.ConsoleAuth

%AI.Policy.Discovery

tool 呼出ごとに自動監査ログ

Provider · ConfigStore 3

④ %AI.Provider

%AI.ConfigStore.LLMDescriptor

%AI.ConfigStore.MCPDescriptor

@{config:AI.LLM.openai} · secret 解決

MCP 1

⑥ %AI.MCP.Service

IRIS 自身を MCP サーバ化。
`/mcp/<name>` に publish すると
Claude Code / ChatGPT 等から
IRIS 内のビジネスロジックを
直接呼び出せる。

LLM 3

%AI.LLM.ContentPart

%AI.LLM.Context

%AI.LLM.Response

LLM と IRIS をつなぐデータモデル

Shell 3

%AI.Shell.Console

%AI.Shell.ConsoleAgent

%AI.Shell.StreamRenderer

IRIS ターミナルで AI と対話

RAG 5

%AI.RAG.Embedding

%AI.RAG.Embedding.FastEmbed

%AI.RAG.Embedding.OpenAI

%AI.RAG.KnowledgeBase

%AI.RAG.VectorStore.IRIS

IRIS 単独で RAG · 外部 Vector DB 不要

Utils 6

%AI.Utils.ConfigStore

%AI.Utils.SettingStore

%AI.Utils.StreamAdapter

%AI.Utils.Template

%AI.Utils.Text

%AI.Utils.WalletStore

ConfigStore 実装 · Wallet · Template

Tools (組み込み) 3

%AI.Tools.FileSystem

%AI.Tools.SQL

%AI.Tools.ShellTools

エージェントが使う既製ツール

Catalog · System · Skills 3

%AI.Catalog

%AI.System

%AI.Skills.Planning

カタログ · システム · プランニング

LT デモで実際に触る 6 クラス

LT デモで部分的に触れる (Provider · MCP)

本日は口頭紹介のみ (RAG · Shell · Utils など)

61 クラスの中から、デモで触る 6 クラス = AI Hub の骨格

AI Hub · 6 機能カタログ

IRIS 2027.1 新機能 — LT デモで全部触ります

COVERAGE 実演 6 / 6

① %AI.Agent + Session

エージェント本体 + 会話/tool call 履歴

```
Class Demo.Agent.PatientSummary Extends %AI.Agent
{
Parameter PROVIDERCONFIG = "@{config:AI.LLM.openai}";
Parameter MODEL = "gpt-4o";
Parameter TOOLSETS = "Demo.ToolSet.PatientSummary";
XData INSTRUCTIONS [ MimeType = text/markdown ]
{ あなたは医療データを要約する日本語アシスタント… }
}
```

LIFECYCLE

`%New → %Init → CreateSession → Chat`

継承 + Parameter + XData INSTRUCTIONS だけで動く

② %AI.Tool

ClassMethod 1 つが tool 1 つ

```
Class Demo.Tool.PatientSearch Extends %AI.Tool
{
/// Patient を FHIR id か氏名（漢字/カナ）で検索する。
ClassMethod SearchPatients(
pQuery As %String = "",
pMaxRows As %Integer = 10
) As %DynamicArray { /* ... */ }
}
```

AUTO-TRANSFORM

docstring → LLM への `description`
signature → `parameter schema`

MessageMap も Parameter DESCRIPTION もいらない

③ %AI.ToolSet (XData)

宣言的集約 · XML で tool 束ねる

```
XData Definition
{
<ToolSet Name="PatientSummary">
<Include Class="Demo.Tool.PatientSearch"/>
<Query Name="RecentEnc">
SELECT * FROM Encounter
WHERE patient = ?
</Query>
</ToolSet>}
```

BONUS

<Query> なら ClassMethod 書かず SQL だけで tool

Interop の MessageMap XData 書いた人の感動ポイント

④ Provider · ConfigStore · Wallet

LLM 設定 + 暗号化キー管理

```
;; 1. Config 登録 (dotted path = AI.LLM.openai)
Do ##class(%ConfigStore.Configuration).Create(
"AI", "LLM", "", "openai",
{"model_provider":"openai", "model":"gpt-4o",
"api_key":"secret://DemoCollection.openai#api_key"})
;; 2. Agent 側は dotted path を参照するだけ
Parameter PROVIDERCONFIG = "@{config:AI.LLM.openai}";
;; → 起動時に Wallet (%Wallet.KeyValue) から api_key を解決
```

BENEFITS

OpenAI / Anthropic / ローカル LLM を **同じ参照**で切替

.env に書く罪悪感から解放

⑤ %AI.Policy (Audit + Auth)

tool 呼出のゲートと監査

```
;; ToolSet の XData に 1 行足すだけ
<ToolSet Name="PatientSummary">
<Policies><Audit Class="%AI.Policy.ConsoleAudit"/></Policies>
<Include Class="Demo.Tool.PatientSearch"/> …
</ToolSet>
;; → tool 呼出ごとに自動ログ出力:
| OK SearchPatients → [{"id":"pt-670b…"}] [6ms]
| OK LatestObservations → {"count":10,…} [7ms]
```

ENTERPRISE READY

どの tool · 何を渡した · 何が返った · 何 ms

医療 AI の「後から説明責任」に即適用可

⑥ %AI.MCP.Service

IRIS 自身を MCP サーバ化

```
Class My.MCP Extends %AI.MCP.Service
{
Parameter SPECIFICATION =
"My.ToolSet.PatientSummary";
}

;; CSP app に publish すると:
POST /mcp/patient-summary
;; 外部 AI クライアントから直接 tool 叩ける
```

BIDIRECTIONAL

IRIS = 「AI を使う」側 + 「AI に提供する」側

Claude Code / ChatGPT から直接 IRIS ロジックを叩く

● **6 ブロック全て** を本日 6 分で実演 — 人間が書くのは日本語プロンプト 1 本だけ

継承 · XData · dotted path · docstring — IRIS らしい作法のまま

AI エージェントは、AI に作らせる

IRIS AI Hub × iris-agentic-dev × Claude Code — 3つの柱の交差点

PHASES steps 8

柱① IRIS AI Hub
%AI.Agent / Tool / ToolSet / Provider
61 クラス · 2027.1

柱② iris-agentic-dev
MCP サーバ · 多 AI クライアント対応
34 skills · v1.6.2

柱③ Claude AI コーディング
自然文プロンプト → 自己修正ループ
0 人間が書く行数

ARCHITECTURE · 3 柱の交差

発表者
自然言語で依頼

Claude Code
Claude Sonnet 4.7 + MCP

AI コーディング ループ

1. プロンプト解釈
2. skill_describe で API 確認
3. iris_doc put でクラス書込
4. iris_compile
5. エラー時: AI が自己修正
6. iris_execute で動作検証

人間が書くもの: 日本語プロンプト 1 本

iris-agentic-dev
MCP Server · v1.6.2

コード	iris_doc / compile / symbols
データ	iris_execute / iris_query
連携	iris_production / interop_query
知識	skill_describe / docs_introspect

BUNDLED SKILLS 34

- iris-ai-hub 2027.1 AI Hub
- objectscript-guardrails
- iris-sql
- ensemble-production
- ...他 30

新技術の API 知識を AI 側に供給

IRIS for Health 2027.1
HSCUSTOM · Advanced Server · 2026-10-21 build

AI build 103

① %AI.Agent + Session
`Demo.Agent.PatientSummary`
%New → %Init → CreateSession → Chat
PROVIDERCONFIG = @{config:AI.LLM.openai}

③ %AI.ToolSet (XData)
`Demo.ToolSet.PatientSummary`
<Include> ×2 + <Policies><Audit>
②docstring → tool spec 自動変換

④ ConfigStore + Wallet
`AI.LLM.openai`
secret://DemoCollection.openai#api_key
%Wallet.KeyValue (暗号化保管)

⑤⑥ Policy + MCP.Service
`%AI.Policy.ConsoleAudit`
tool 呼出ごとに自動監査ログ
/mcp/ で外部AIから被叩き可*

FHIR R4 REPOSITORY · /csp/healthshare/hscustom/fhir/r4
Clinosim v0.6.4 JP cohort · 2026-01〜03 · seed=42

PATIENTS	OBSERVATIONS	ENCOUNTERS	MED ADMIN
53	8,111	104	897

合計 13,367 リソース · LOINC + MHLW YJコード + 日本人氏名/住所

OpenAI
gpt-4o
function calling

TIMELINE · 8 フェーズ

1. **プロンプト投入** [柱③] 日本語で指示
2. **skill 自動参照** [柱②] iris-ai-hub
3. **put + compile** [柱②+①] クラス4本
4. **クラス完成** [柱①] docstring→tool
5. **Chat → Tool実行** [柱①] FHIR SQL
6. **Provider呼出** [柱①] secret解決
7. **gpt-4o 応答** 日本語 + 医学的解釈
8. **要約表示** 発表者へ戻る

柱③ Claude AI コーディング　柱② iris-agentic-dev MCP　柱① IRIS AI Hub　LLM Provider (OpenAI 等)

人間の書くもの: 日本語プロンプト 1 本 · 所要時間: 6 分 · 従来比 720x

今、見たやつの「お、すごい」3 連

6 分のライブデモでなぜ「ざわっ」ときたか — 3 つの瞬間を言語化

DEMO ELAPSED
live
6 min

!

3:30 頃

WOW ①

クラスリファレンスを開かない開発

何が起きた

AI が **iris-ai-hub** スキルを
自動参照し、2027.1 の最新 API 書式を取得。

BEFORE	AFTER
半日	0 秒
クラスリファレンス往復 + 試行錯誤	AI 側が最新 API を bundled で保持

PUNCHLINE

新 API ほど効く。

人間が調べても出てこない情報を
AI が最初から持ってる。

`iris_doc · bundled skill 34`

!

5:00 頃

WOW ②

XData の XML 地獄が、宣言的な数行に消えた

BEFORE — 従来 MessageMap XData

```
<MapItem MessageType="Pat.*">
<Method>OnSearch</Method>
<ReplyMessage>...</ReplyMessage>
;; さらに Parameter DESCRIPTION 書いて
```

AFTER — %AI.ToolSet XData Definition

```
<ToolSet Name="PatientSummary">
<Policies><Audit Class="%AI.Policy.ConsoleAudit"/></Policies>
<Include Class="Demo.Tool.PatientSearch"/>
<Include Class="Demo.Tool.ObservationSearch"/>
</ToolSet> ;; たったこれだけ
```

PUNCHLINE

Interop 書いた人の、感動ポイント。

docstring → tool description、signature → schema。
監査ログも <Policies> 1 行で全 tool に。

`%AI.Tool · %AI.ToolSet · %AI.Policy`

!

7:00 頃

WOW ③

AST も ALT も 28。ALT だけ「高値」

AGENT RESPONSE (日本語) — 横山 静子 96 歳

バイタル:
血圧: 141/85 mmHg **(収縮期 高値)**
脈拍: 87 /min

検査値:
AST: 28 U/L (基準 13–30)
ALT: 28 U/L (基準 7–23) **(高値)**

PUNCHLINE

単なるデータ取得じゃない。

FHIR の基準範囲と interpretation を読んで、
異常値だけを AI が拾い出す。

*人間が書いたコードは **1 行もなし**。*

`%AI.Agent + gpt-4o (約 7 秒)`

RECAP · 6 MIN LIVE DEMO

人間が書いたのは — 日本語プロンプト 1 本だけ。

AI が docs を引き · XData を書き · コンパイルを通し · 臨床判断まで乗せた。

HUMAN CODE
0
lines (except prompt)

従来の IRIS 開発 vs AI 支援

Patient 要約エージェント (4 クラス + Policy) をゼロから作る実測値

開発者がやっていた作業	従来 (Studio / VS Code) ドキュメント往復 + 手打ち	AI 支援 (Claude Code + iris-agentic-dev)
%AI.* の 61 クラス API を クラスリファレンスで引く	半日 〜 1 日	**0 秒 (bundled skill)**
ConfigStore + Wallet の試行錯誤 (RebuildRegistry 等のハマり)	半日	**1 回で通る**
%AI.Tool + ToolSet の XData 構文確認 (XML 書式)	1 日	**数十秒**
FHIR Observation JSON extract コード ($Piece ネスト地獄)	半日	**AI が書く**
日本語 SystemPrompt のチューニング	半日 〜 1 日	**1 プロンプトで指定**
%AI.Policy (Audit) の統合	半日	**プロンプトに 1 行**
(Studio 派刺し込み) DTL / Rule / Routing クリック作業	半日 〜 1 日	**「○○を△△にマッピングして」で XData 生成**
合計	**3 - 4 日** 8 時間 × 3 日 = 24h	**6 分** プロンプト 1 本 + 待ち時間

EFFICIENCY MULTIPLIER

720x

24h → 6min

新しい技術が出た時こそ AI 支援の効果が最大。

「ドキュメント追いかける」部分がほぼ 0 — 既存改修より、
「まだ誰も触ってない API を 1 人目として触る」時こそ効く。

— IRIS に新機能が入るたび、追従コストが実質ゼロ。

計測条件: Patient 要約エージェント (Demo.Agent.PatientSummary + Tool/ToolSet + Policy + Provider) · Clinosim v0.6.4 JP · OpenAI gpt-4o

IRIS 開発者ツール × 能力マトリクス

Studio ／ VS Code + IRIS 拡張 ／ + Claude Code + iris-agentic-dev

✅ 得意　➖ 部分的　❌ 不可

2026-10 時点 / IRIS 2027.1

開発者がやりたい作業	Studio Windows / 統合ビジュアル	VS Code + IRIS 拡張 クロスプラットフォーム / モダン	+ MCP クライアント + iris-agentic-dev Claude Code / Codex / Cursor 等 (本日は Claude Code で実演)
A. Interoperability ビジュアル編集 — Studio の牙城			
DTL エディタ (ドラッグ&ドロップで変換)	✅	❌	✅ XData を textual に生成 / 編集
Business Rule エディタ	✅	❌	✅ RuleDefinition XData を直接編集
Message Routing (Router Rule)	✅	❌	✅ Production.cls XData を編集
HL7 Schema エディタ	✅	❌	✅ hl7_schema_inspect で即引き
Lookup Table / BPL / Record Mapper	✅	❌	✅ iris_lookup_manage / XML 編集
B. モダンエディタ体験 — VS Code の牙城			
マルチカーソル / モダン検索・置換 (regex)	❌	✅	✅ VS Code の機能をそのまま併用
サイドバイサイド diff / Merge Editor	❌	✅	✅ compare_document で CLI diff ☒も可
統合デバッガ (ObjectScript step-through)	✅	➖	✅ iris_debug / iris_get_log で補助
C. 開発基盤 — git / クロスプラットフォーム / リモート			
Mac / Linux で動く	❌	✅	✅ Claude Code 自体がクロスプラットフォーム
ファイルベース git ワークフロー	➖	✅	✅ iris_source_control で統合制御
Remote Development (SSH / Container)	❌	✅	✅ MCP TOML で接続先切替
D. AI 支援 — 従来両方できなかった新領域			
クラスリファレンスを開かずに API を引く	❌	❌	✅ **iris_doc + bundled skill 34 本**
新 API 追従 (例: %AI.* 2027.1 の 61 クラス)	❌	❌	✅ **skill 自動参照で即日利用**
Production / DTL / Rule を自然言語で生成・編集	❌	❌	✅ **iris_production + XData 生成**
社内規約・社内フレームワーク準拠	❌	❌	✅ **CLAUDE.md + 自作 skill**

結論 — Studio vs VS Code の宗派論争より上の階層

- **Studio 派ほど効果大** — DTL / Rule / Routing のクリック作業は全部 XData の XML。AI が最も得意な領域と「Studio の牙城」がちょうど重なる。
- **VS Code 派は役割分担** — 補完・ハイライト・ServerSync は IRIS 拡張。横断的な調査・リファクタ・Production 編集は Claude Code。
- **Claude Code は CLI** — Studio 横の別ターミナル、VS Code の統合ターミナル、どちらでも OK。 **エディタ乗り換え不要**。